

Software Requirements Specification

MEA-V2G / FreeV2G

Reference Implementation for ISO 15118 EV/EVSE

Ruslee Sutthaweeikul

December 20, 2025

Contents

1	Introduction	3
1.1	Purpose	3
1.2	Scope	3
1.3	Definitions and Acronyms	3
2	Overall Description	4
2.1	Product Perspective	4
2.2	System Functions	4
2.3	Use Case Diagram	4
2.4	System Architecture	4
2.5	Core Components	4
2.5.1	Whitebeet Component	5
2.5.2	EV Component	5
2.5.3	EVSE Component	6
2.5.4	OCPP Component	6
2.6	Charger Component	7
2.6.1	Implementing a Custom Charger	7
2.7	Battery Component	9
2.8	Default Charging Routine	9
2.8.1	Default Schedule	9
2.8.2	Physics Simulation Parameters	9
2.9	Project File Structure	9
3	Specific Requirements	11
3.1	External Interfaces	11
3.1.1	Communication Interfaces	11
3.1.2	User Interfaces	11
3.2	Functional Requirements	11
3.2.1	FR-01: Protocol Negotiation	11
3.2.2	FR-02: Authorization	11
3.2.3	FR-03: Charging Control	12
3.2.4	FR-04: OCPP Integration	12
3.3	Communication Flow	12
3.3.1	OCPP Sequence Flow	12
3.3.2	CSMS-EVSE-EV Interaction	13
3.4	EV State Machine	13
3.5	EVSE State Machine	14
3.6	State Synchronization	15
3.7	OCPP Connector State Machine	17
3.7.1	State Transition Logic	17

3.7.2	EVSE to OCPP State Mapping	18
4	Configuration and Deployment	20
4.1	Command Line Interface (CLI)	20
4.2	JSON Configuration	20
4.2.1	EV Configuration (ev.json)	20
4.2.2	EVSE Configuration (evse.json)	20

Chapter 1

Introduction

1.1 Purpose

The purpose of this document is to define the software requirements for the MEA-V2G (FreeV2G) project. This system serves as a reference implementation for controlling the 8 devices WHITE-beet-EI (EVSE) and WHITE-beet-PI (EV) ISO 15118 modules using an Ethernet host control interface (HCI).

1.2 Scope

The MEA-V2G software encompasses:

- Parsing of the protocol used to communicate with the WHITE-beet modules.
- Simulation of EVSE and EV behaviors.
- Handling of charging parameters (Voltage, Current, Power).
- Implementation of basic Control Pilot (CP) and SLAC sequences.
- High-level V2G communication management.
- Integration with OCPP 2.0.1 for backend communication.

1.3 Definitions and Acronyms

EV Electric Vehicle

EVSE Electric Vehicle Supply Equipment (Charging Station)

HCI Host Control Interface

SLAC Signal Level Attenuation Characterization

V2G Vehicle-to-Grid

PNC Plug & Charge

EIM External Identification Means

OCPP Open Charge Point Protocol

CSMS Charging Station Management System

Chapter 2

Overall Description

2.1 Product Perspective

The software interacts directly with the WHITE-beet hardware modules via Ethernet or SPI. It acts as the "Host" controller, managing the logic while the WHITE-beet module handles the low-level PLC (Power Line Communication) and ISO 15118 framing.

2.2 System Functions

The system enables the following core functions:

1. **Control Pilot (CP) implementation:** Detects EV plugin and manages duty cycles.
2. **SLAC:** Automates the matching process between EV and EVSE at the physical layer.
3. **V2G Session Management:** Handles Service Discovery, Parameter Discovery, and Authorization.
4. **Charging Loop:** Monitoring and control of the active energy transfer.
5. **OCPP Communication:** Interface with a CSMS via OCPP 2.0.1 for remote control and status monitoring.

2.3 Use Case Diagram

The following diagram illustrates the primary actors and use cases for the MEA-V2G system.

2.4 System Architecture

The system obeys a modular object-oriented architecture. The core logic handles the state machines for EV and EVSE, while the ‘Whitebeet’ class abstracts the hardware communication.

2.5 Core Components

The system comprises the following key software components:

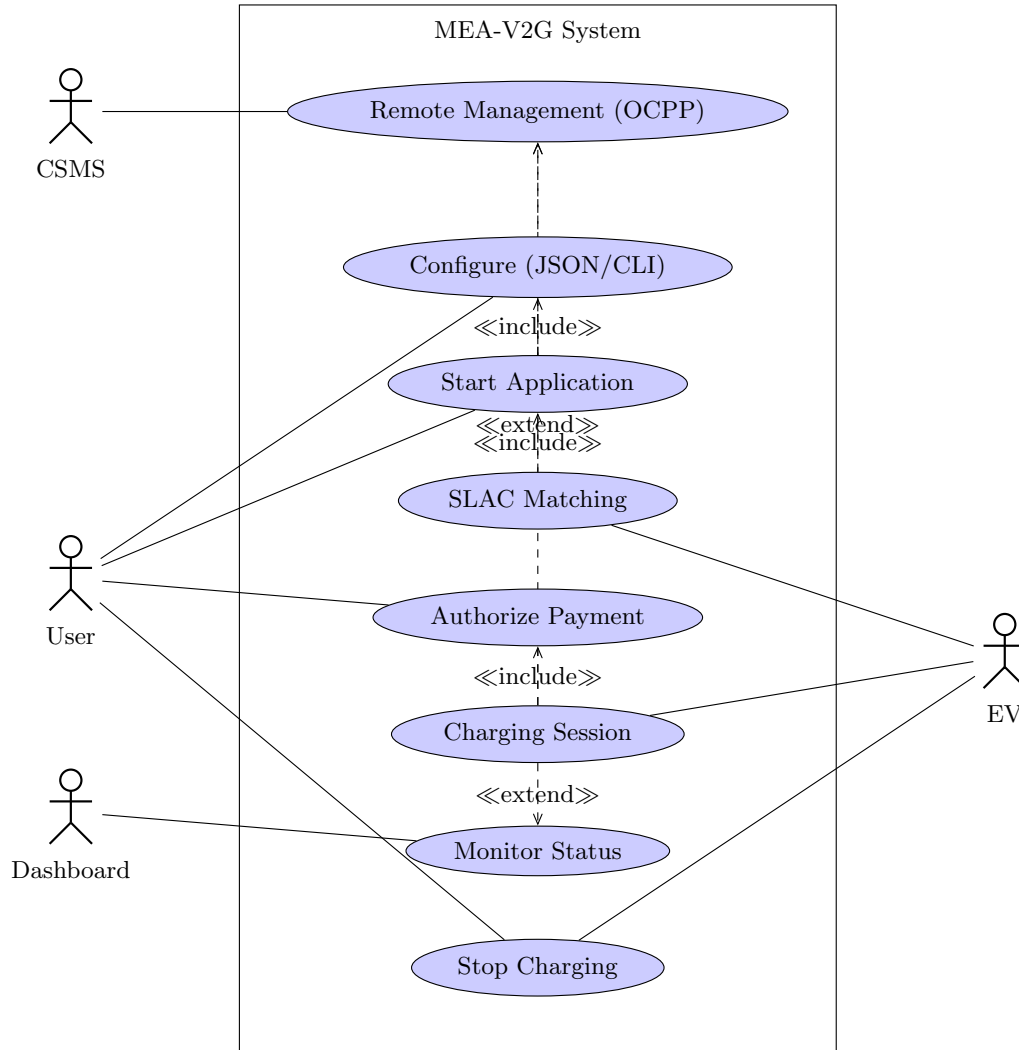


Figure 2.1: MEA-V2G High-Level Use Case Diagram

2.5.1 Whitebeet Component

The **Whitebeet** class serves as the Hardware Abstraction Layer (HAL). It manages the low-level communication with the physical ISO 15118 module via Ethernet or SPI.

- **Packet Framing:** Handles the encapsulation of Host Control Interface (HCI) commands.
- **Event Dispatching:** Receives raw data from the hardware and dispatches events (e.g., EV_CONNECTED, V2G_MESSAGE) to the higher-level logic.
- **Service Management:** Exposes methods to control Control Pilot (CP) state, SLAC, and V2G messaging.

2.5.2 EV Component

The **EV** class is the central controller for the client side. It orchestrates the charging session from the vehicle's perspective.

- **State Machine:** Implements the ISO 15118 client flow (Session Setup → Charge Parameter Discovery → Power Delivery).

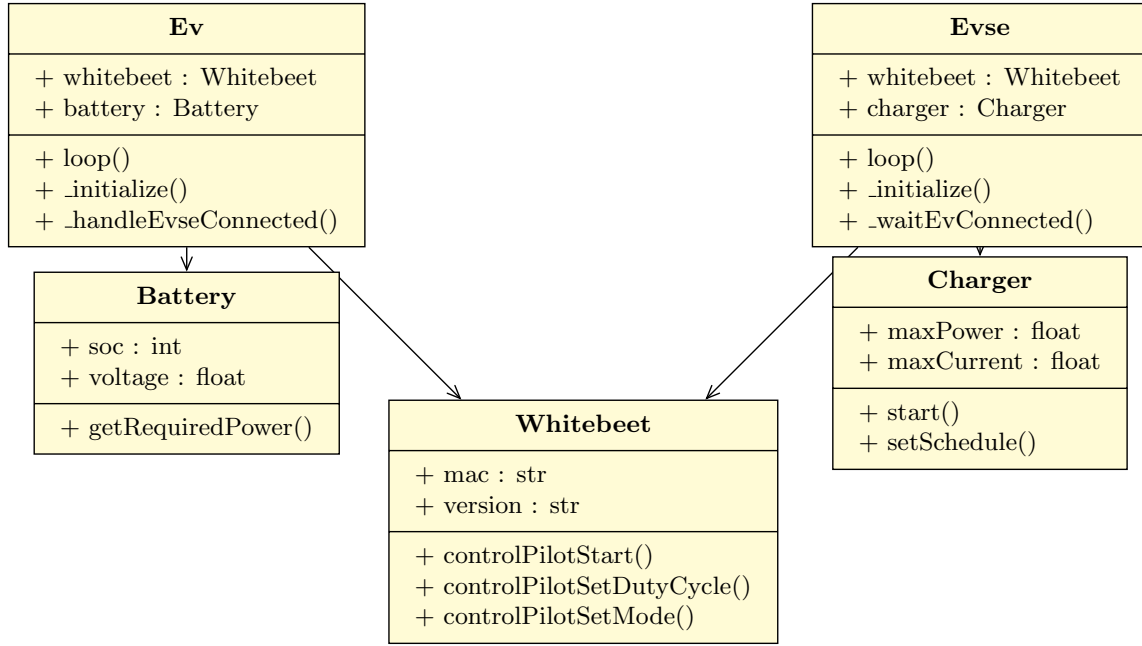


Figure 2.2: MEA-V2G Simplified Class Diagram

- **Hardware Aggregation:** Owning instance of **Whitebeet** (for comms) and **Battery** (for energy simulation).

2.5.3 EVSE Component

The **EVSE** class is the central controller for the station side. It manages the server-side logic and power delivery authorization.

- **State Machine:** Implements the ISO 15118 server flow, including Service Discovery and Plug & Charge authorization.
- **Hardware Aggregation:** Owning instance of **Whitebeet** (for comms) and **Charger** (for power delivery simulation).
- **Scheduling:** Manages **SASchedules** based on configuration.

2.5.4 OCPP Component

The system integrates an OCPP 2.0.1 client to enable remote management.

- **OcppInterface:** Adapts the `ocpp` library's **ChargePoint** class to the MEA-V2G architecture. It maps OCPP commands (e.g., **RequestStartTransaction**) to local **Charger** actions.
- **OcppWorker:** A dedicated thread that manages the `asyncio` event loop and WebSocket connection to the CSMS, ensuring non-blocking operation alongside the main application loop.

2.6 Charger Component

The **Charger** class performs a physical simulation of the power electronics, ensuring realistic behavior during the charging session. Its key functions include:

- **Output Ramping:** Instead of instantaneous jumps, the charger simulates the physical "slewing" of voltage and current over time. It uses defined ΔU and ΔI rates to ramp the output toward the target setpoint.
- **Limit Enforcement:** It strictly enforces hard safety limits for the system (e.g., `evse_max_current`, `evse_max_power`), protecting the modeled hardware.
- **Setpoint Control:** The EV provides dynamic target values (*Target Voltage*, *Target Current*), which the charger attempts to follow within its safety constraints.
- **Fault Detection:** The class includes methods like `isVoltageLimitExceeded()` and `isPowerLimitExceeded()` to trigger immediate session termination if operating conditions are violated.

The **Charger** class implements the **ChargerInterface** abstract base class. This design pattern allows the simulation module to be easily swapped with a real hardware driver (e.g., a **CanCharger** class) that communicates with physical power electronics via CAN bus or other protocols, while maintaining a consistent API for the EVSE logic.

2.6.1 Implementing a Custom Charger

To integrate specific power electronics hardware, developers must create a subclass of **ChargerInterface**. This ensures the EVSE logic remains decoupled from the physical layer.

A custom implementation must override the abstract methods to map high-level commands to hardware-specific protocols. The following sections detail the requirements for each group of methods.

Lifecycle Management

These methods control the high-level operational state of the power electronics.

start() Called when the EVSE initializes or prepares for a session. It should initialize hardware communication (e.g., open CAN sockets). **Note:** This method should **not** automatically close the output contactors; that is handled by the state machine after safe voltage matching.

stop() **Critical Safety Method.** Called to terminate a session or during error conditions. It must immediately disable power output, open contactors, and return the hardware to a safe idle state (0V/0A).

Control Loop Methods

These methods are called cyclically (typically every 10-100ms) during the active charging phase.

setEvTargetVoltage(val) Receives the EV's requested voltage setpoint (in Volts). The implementation must transmit this request to the power supply unit (PSU).

setEvTargetCurrent(val) Receives the EV's requested current setpoint (in Amps). The implementation must ensure the PSU imposes this current limit.

Return Value: These methods must return **True** if the command was successfully queued or sent to the hardware, and **False** if the request triggers a local hardware fault or limit violation.

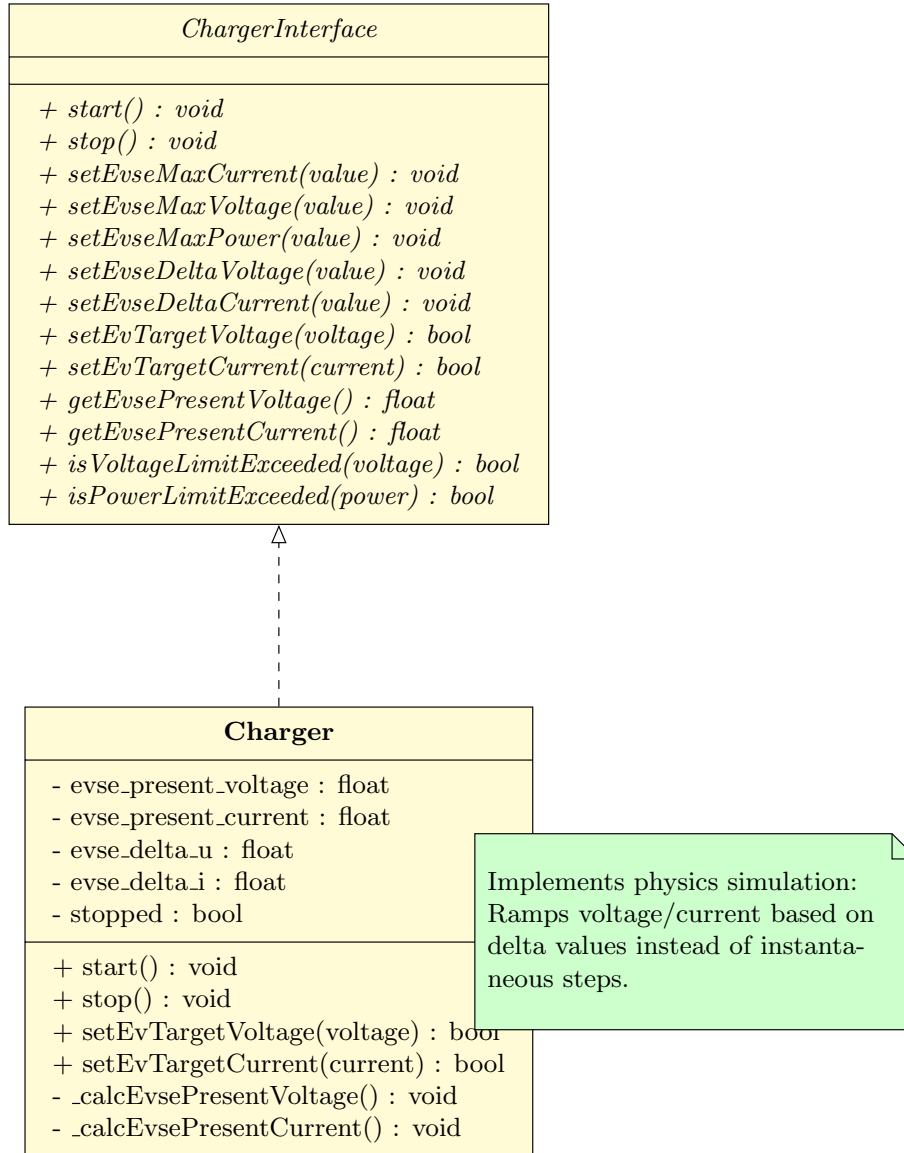


Figure 2.3: Charger Interface and Implementation

Telemetry Methods

The EVSE state machine relies on accurate feedback to perform safety checks (e.g., Welding Detection) and process steps (e.g., Pre-Charge).

getEvsePresentVoltage() Must return the *measured* voltage at the DC output terminals. Do not return the internal setpoint; the actual value is required to verify contactor states.

getEvsePresentCurrent() Must return the *measured* current flowing to the EV.

Configuration Accessors

The interface includes getters and setters for electrical limits (e.g., **setEvseMaxCurrent**, **getEvseMaxPower**).

- For a **Simulation** (like **Charger.py**), these methods simply store and retrieve variables to emulate different hardware capabilities.

- For **Physical Hardware**, the setters (e.g., 'setEvseMaxCurrent') should theoretically configure the hardware's internal safety latches if supported. The getters must return the valid limits to be advertised to the EV during the *Charge Parameter Discovery* phase.

2.7 Battery Component

The **Battery** class provides a logical model of the EV's energy storage, managing State of Charge (SOC) and physical limits.

- **Energy Tracking:** Maintains the battery capacity (Wh) and current energy level. SOC is dynamically calculated as $\frac{Current\ Level}{Capacity} \times 100$.
- **Charging Process:** The `tickSimulation()` method accumulates energy over time intervals based on the instantaneous input voltage and current ($E = V \times I \times \Delta t$).
- **Configurable Limits:** Supports defining maximum Voltage, Current, and Power constraints specific to the chosen Energy Transfer Mode (AC vs. DC).
- **Status Reporting:** Continuously updates the SOC and charging status ('is_full', 'is_charging'), which are reported back to the EVSE via ISO 15118 messages.

2.8 Default Charging Routine

In the absence of an external configuration file, the system defaults to a hardcoded "Simple Charging Routine" defined in `Application.py`. This routine ensures the EVSE behaves predictably during initial testing or standalone operation.

2.8.1 Default Schedule

The default charging profile is a time-dependent power curve that simulates a decreasing power availability (e.g., grid balancing or solar curve):

- **0 to 30 minutes:** Maximum Power 25,000 W.
- **30 to 60 minutes:** Maximum Power reduced to 18,750 W.
- **Over 60 minutes:** Maximum Power further reduced to 12,500 W.

2.8.2 Physics Simulation Parameters

To simulate realistic hardware behavior, the **Charger** class uses specific delta parameters to ramp voltage and current rather than applying instantaneous steps:

- **Delta Voltage:** 0.5 V per tick.
- **Delta Current:** 0.05 A per tick.
- **Max Limits:** 400 V / 100 A / 25 kW.

2.9 Project File Structure

The following table summarizes the key files in the repository and their specific purposes within the architecture:

File Name	Description
Application.py	The main entry point. Parses command-line arguments and initializes the EV or EVSE instance based on the selected role.
Ev.py	Implements the Electric Vehicle (client) logic, including the ISO 15118 state machine and session management.
Evse.py	Implements the Charging Station (server) logic, managing service discovery, authorization, and power delivery.
Whitebeet.py	The Hardware Abstraction Layer (HAL). Handles low-level communication with the Codico Whitebeet module.
Charger.py	A physics simulation class for the EVSE's power electronics. Simulates voltage/current ramping and enforces safety limits.
ChargerInterface.py	An abstract base class defining the standard API for charger implementations, enabling pluggable drivers (Simulation vs. CAN).
Battery.py	A logical model of the EV battery, tracking State of Charge (SOC) and energy capacity.
EthernetAdapter.py	Driver for communicating with the Whitebeet module via a network socket.
SpiAdapter.py	Driver for communicating with the Whitebeet module via the SPI bus (e.g., on Raspberry Pi).
FramingInterface.py	Defines the packet structure and framing rules for the Host Control Interface (HCI) protocol.
api_server.py	A lightweight Flask server that exposes internal state (like SOC, current power) via a REST API for monitoring.
gui_launcher.py	A graphical utility to easily launch Application.py without using the command line.
ev.json / evse.json	Configuration files defining capabilities, limits, and schedules for the respective roles.

Chapter 3

Specific Requirements

3.1 External Interfaces

3.1.1 Communication Interfaces

- **Ethernet:** Main control interface for sending HCI commands and receiving notifications.
- **SPI:** Alternative control interface (e.g., for Raspberry Pi integration).
- **Ocpp Interface:** WebSocket connection (JSON over WebSockets) to a central management system (CSMS).

3.1.2 User Interfaces

- **CLI (Command Line Interface):** Primary mode of operation. Outputs session logs and accepts interactive input (e.g., authorization "yes/no").
- **GUI:** A graphical launcher ('gui_launcher.py') provided for configuration and startup.
- **API:** A REST API (default port 5000) for real-time status monitoring (e.g., for Grafana dashboards).

3.2 Functional Requirements

3.2.1 FR-01: Protocol Negotiation

The system shall support negotiation of ISO 15118 protocol versions.

- The EVSE shall advertise supported protocols.
- The EV shall select a mutually supported protocol (e.g., DIN SPEC 70121 or ISO 15118-2).

3.2.2 FR-02: Authorization

The system shall support multiple authorization modes:

- **EIM (External Identification Means):** Manual authorization via console input.
- **PnC (Plug & Charge):** Automated certificate-based authorization.

3.2.3 FR-03: Charging Control

During the charging loop, the system shall:

- Exchange charging parameters (SOC, Target Voltage, Target Current).
- Terminate the session upon "Stop Charging" request from either side.

3.2.4 FR-04: OCPP Integration

The system shall implement OCPP 2.0.1 features:

- Connect to a CSMS via WebSocket.
- Send `BootNotification` and `Heartbeat` messages.
- Accept `RequestStartTransaction` and `RequestStopTransaction` commands from the CSMS.
- Support `SetVariables` to configure EVSE limits remotely.

3.3 Communication Flow

The following sequence diagram depicts the simplified ISO 15118 charging loop interaction between the EV and EVSE, which is the core logical flow of the application.

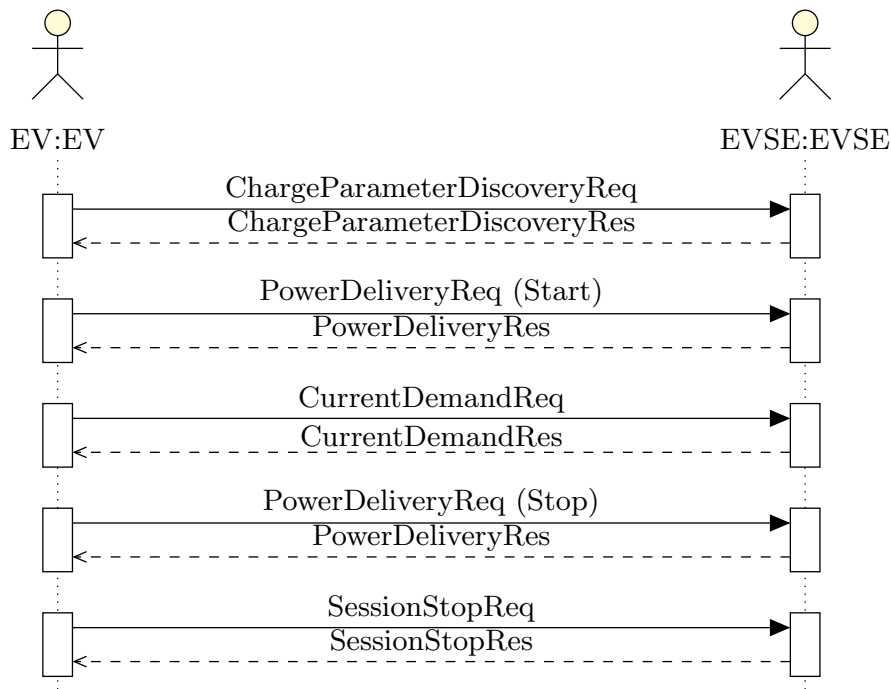


Figure 3.1: ISO 15118 Charging Loop Sequence

3.3.1 OCPP Sequence Flow

The following diagram illustrates the interaction between the EVSE (Charge Point) and the CSMS (Charging Station Management System) via OCPP 2.0.1.

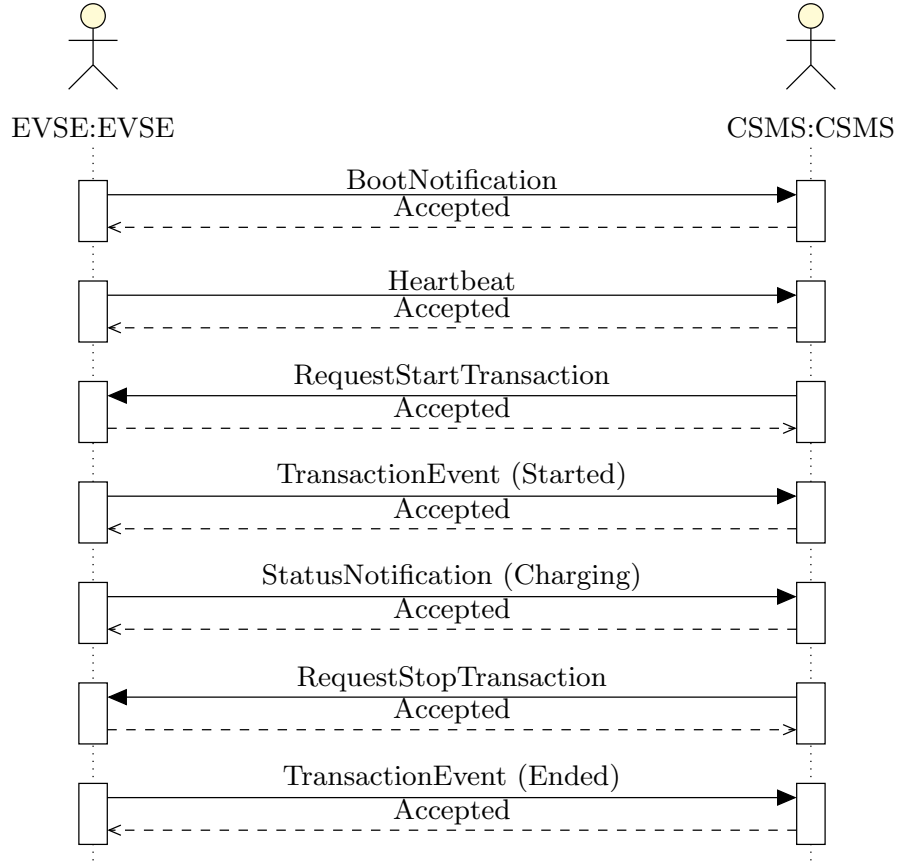


Figure 3.2: OCPP 2.0.1 Interaction Sequence

3.3.2 CSMS-EVSE-EV Interaction

This diagram focuses on the 3-tier communication flow involving the management system, the charging station, and the vehicle. It illustrates a scenario where a remote command initiates a session with smart charging limits.

3.4 EV State Machine

The EV internal logic follows a distinct state machine to manage the charging session lifecycle, as observed in the implementation.

Init Initial state where the EV controller is started and hardware interfaces (SPI/Ethernet) are initialized.

SessionStarted The V2G session has been established. The EV has successfully matched with the EVSE via SLAC.

CableCheck The EV waits for the EVSE to perform the cable check (isolation test) to ensure safety.

PreCharging Pre-charge phase where the EV matches its battery voltage to the DC bus voltage.

Charging The main state where energy transfer occurs. The EV sends *CurrentDemand* requests in a loop.

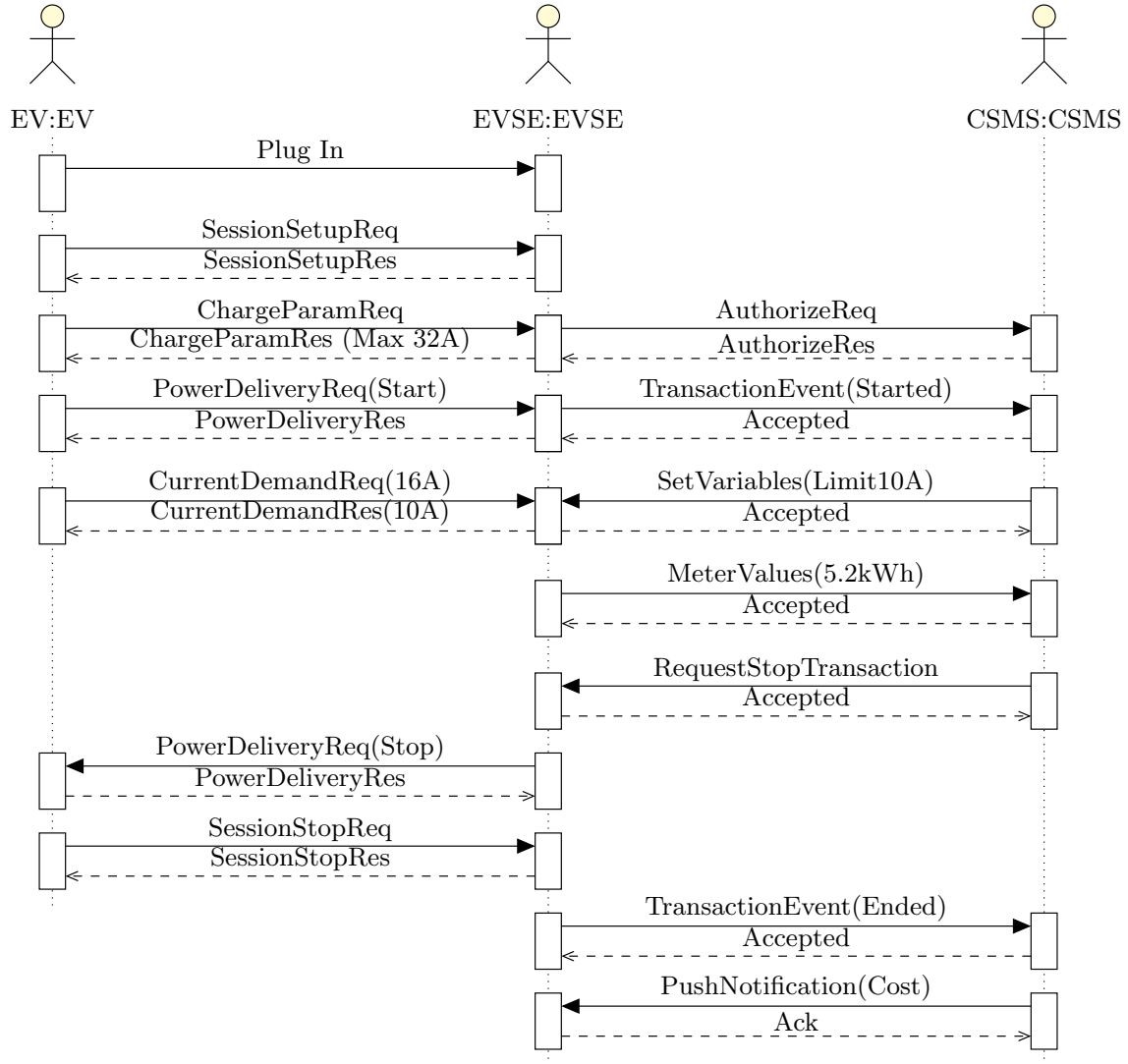


Figure 3.3: 3-Tier (CSMS-EVSE-EV) Sequence Flow

ChargingStopped The session is terminated, either normally (target SOC reached) or due to an error.

3.5 EVSE State Machine

The EVSE logic complements the EV, acting as the server component that handles session setup, authorization, and power delivery control.

Init Initialization of the Whitebeet module services (CP, SLAC, V2G).

Wait The EVSE waits for an EV to physically plug in (Control Pilot State B).

SLAC Signal Level Attenuation Characterization process to pair the EV and EVSE.

Session High-level communication session established; protocol negotiation occurs.

Authorization Verifying if the EV is authorized to charge (via EIM or Plug & Charge).

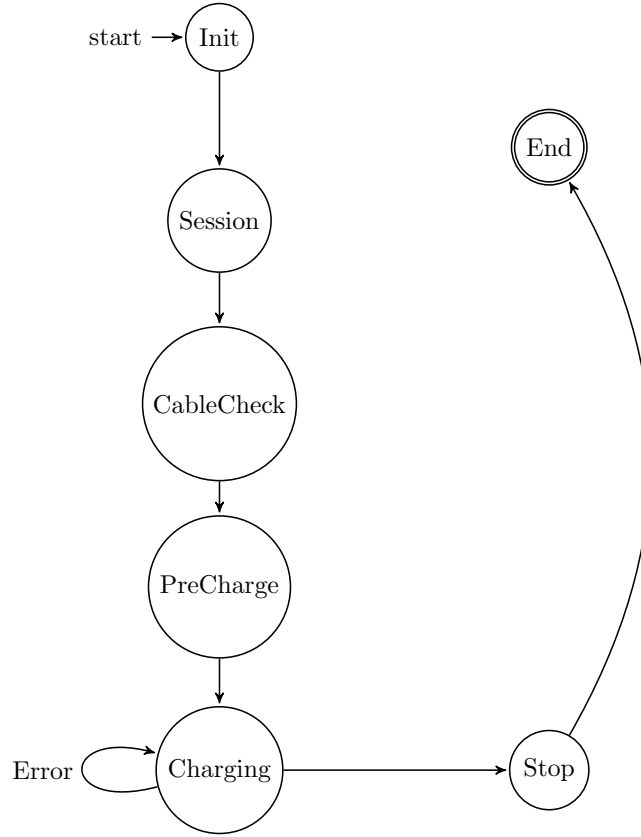


Figure 3.4: EV State Machine

Params Exchanging charge parameter limits (Maximum Voltage, Current, Power limits).

Cable Performing cable isolation check to ensure no faults exist.

PreCharge Adjusting DC output voltage to match the EV's reported battery voltage.

Charge Active energy transfer loop responding to the EV's current demand.

Stop Session termination and welding detection sequence.

3.6 State Synchronization

The charging process relies on the tight synchronization between the EV and EVSE states. Figure 3.6 illustrates the parallel state progression and key synchronization points.

Discovery Control Pilot signal detection (State A $\rightarrow$ B).

Setup SLAC matching and V2G Session Setup.

Safety Both sides must confirm safety parameters during Cable Check.

Sync Voltage levels must be synchronized during Pre-Charge before the contactors close.

Power The Charging Loop is a lock-step process where the EV requests and the EVSE delivers.

Term Coordinated shutdown to ensure contactors open safely (Welding Detection).

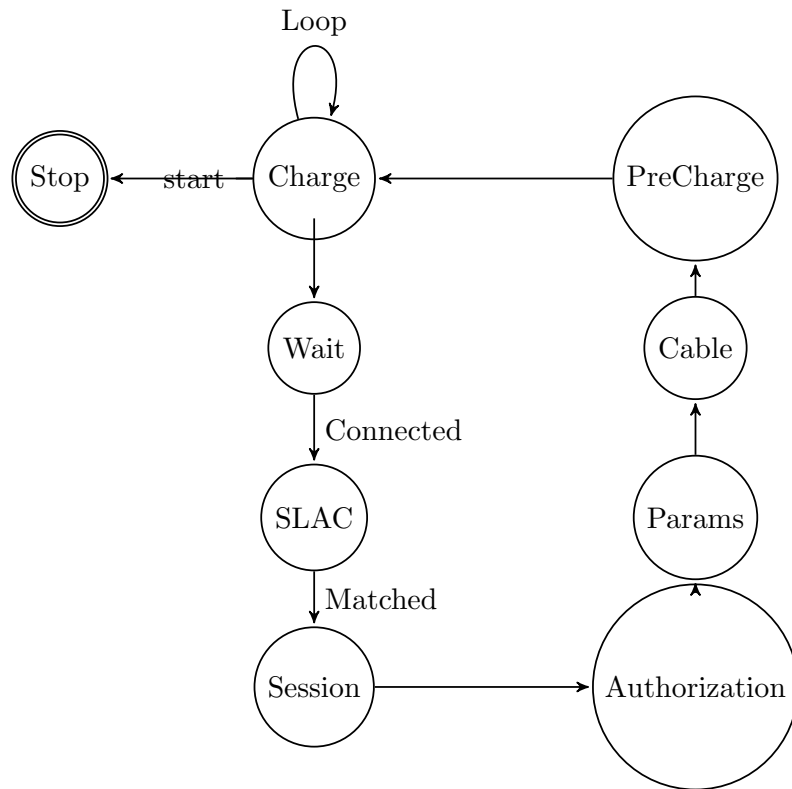
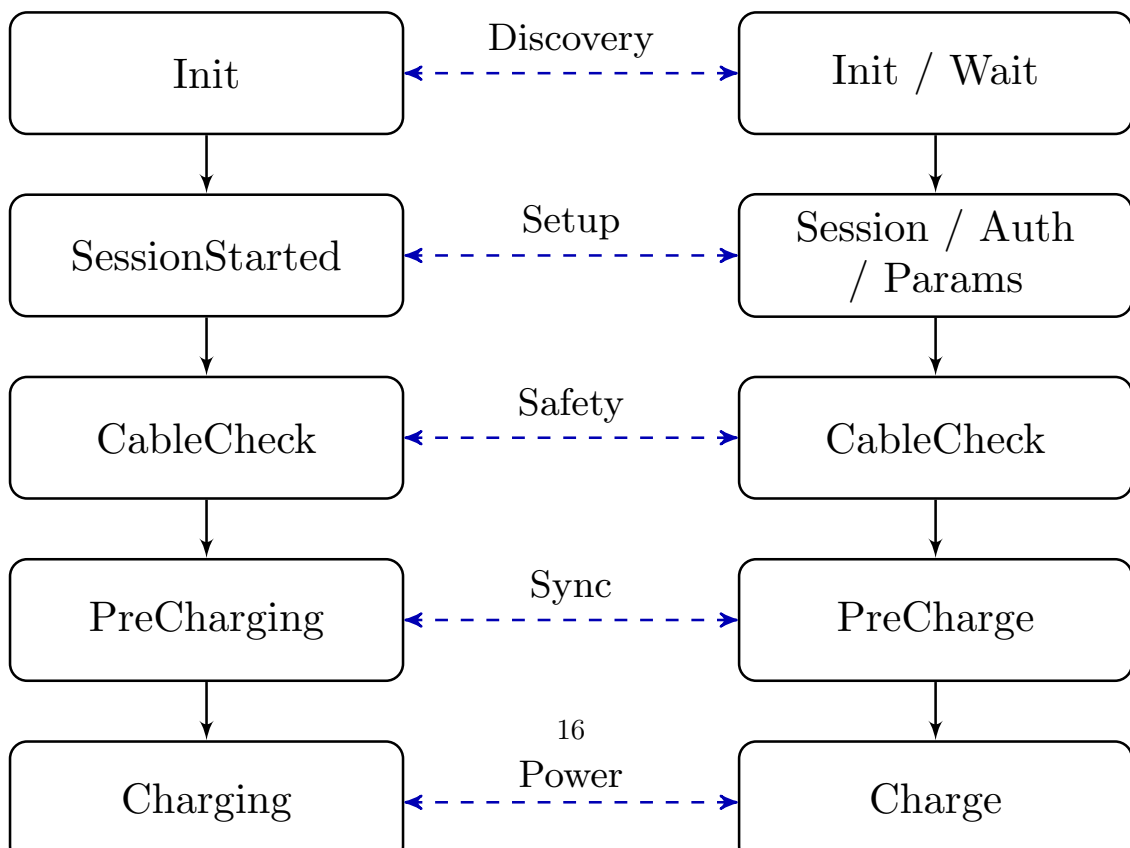


Figure 3.5: EVSE State Machine

EV

EVSE



3.7 OCPP Connector State Machine

The EVSE maintains a logical state for each connector, which is reported to the CSMS via Status Notification messages. This state machine abstracts the underlying ISO 15118 physical processes.

Available The connector is free and ready for a new session.

Preparing An EV is plugged in or an authorization process is pending.

Charging Active energy transfer is in progress.

SuspendedEV Charging is paused by the EV (e.g., battery full).

SuspendedEVSE Charging is paused by the EVSE (e.g., smart charging limit).

Finishing The session has stopped, waiting for the plug to be removed.

Reserved The connector is reserved for a specific user.

Unavailable The connector is operative but not available for use (e.g., maintenance).

Faulted A critical error has occurred.

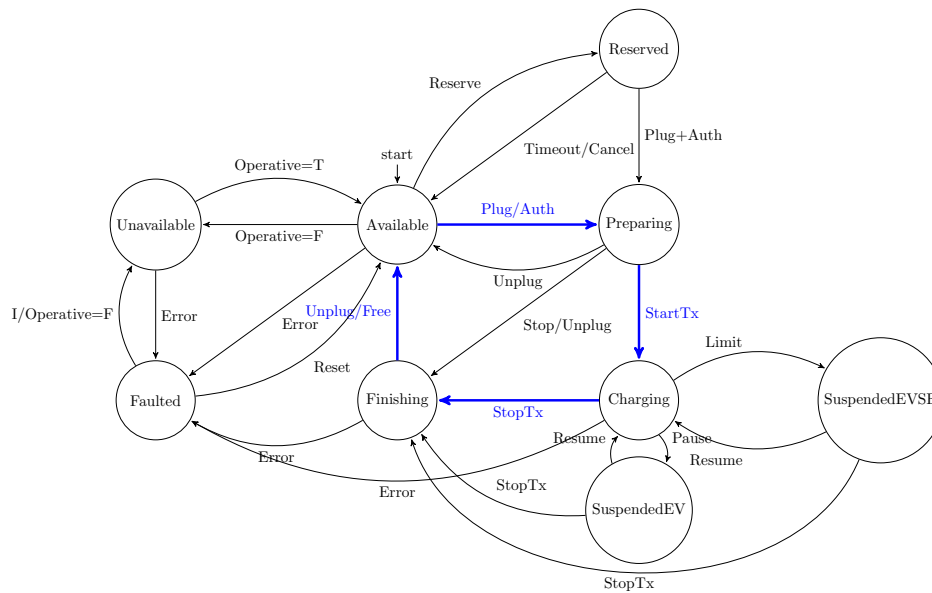


Figure 3.7: OCPP Connector Status State Machine

3.7.1 State Transition Logic

The following table details the communication packets exchanged between the EV, EVSE, and CSMS that trigger the transitions in the OCPP Connector State Machine.

Transition	Physical Trigger	ISO 15118 (EV-EVSE)	OCPP (EVSE-CSMS)
<i>Normal Operation</i>			
Available → Preparing	Plug In (CP $A \rightarrow B$)	SLAC Match, SessionSetupReq	StatusNotification(Preparing)
Preparing → Charging	Power Delivery Start (CP $B \rightarrow C$)	PowerDeliveryReq(Start), CurrentDemandReq	TransactionEvent(Started), StatusNotification(Charging)
Charging → Finishing	Session Stop (CP $C \rightarrow B$)	PowerDeliveryReq(Stop) or SessionStopReq	TransactionEvent(Ended), StatusNotification(Finishing)
Finishing → Available	Unplug (CP $B \rightarrow A$)	N/A	StatusNotification(Available)
<i>Suspended States</i>			
Charging → SuspendedEV	EV pauses (e.g. max SOC)	CurrentDemandReq (Target=0A)	StatusNotification(SuspendedEV)
SuspendedEV → Charging	EV resumes	CurrentDemandReq (Target>0A)	StatusNotification(Charging)
Charging → SuspendedEVSE	EVSE limits power	CurrentDemandRes (MaxCurrent=0A)	StatusNotification(SuspendedEVSE)
SuspendedEVSE → Charging	EVSE lifts limit	CurrentDemandRes (MaxCurrent>0A)	StatusNotification(Charging)
Suspended → Finishing	Stop during pause	SessionStopReq	TransactionEvent(Ended), StatusNotification(Finishing)
<i>Reservation & Availability</i>			
Available → Reserved	N/A	N/A	ReserveNowReq, StatusNotification(Reserved)
Reserved → Preparing	Plug In + Auth	SessionSetupReq	StatusNotification(Preparing)
Reserved → Available	Timeout / Cancel	N/A	CancelReservationReq, StatusNotification(Available)
Available → Unavailable	Operator Command	N/A	ChangeAvailabilityReq, StatusNotification(Unavailable)
Unavailable → Available	Operator Command	N/A	ChangeAvailabilityReq, StatusNotification(Available)
<i>Exception Layout</i>			
Preparing → Available	Unplug (Invalid Session)	SessionStopReq	StatusNotification(Available)
Any → Faulted	Error / Fault	Welding Detection Fail, etc.	StatusNotification(Faulted)
Faulted → Available	Reset / Recovery	N/A	ResetReq, StatusNotification(Available)
Faulted → Unavailable	Permanent Fault	N/A	ChangeAvailabilityReq, StatusNotification(Unavailable)

3.7.2 EVSE to OCPP State Mapping

The following table defines how the internal ISO 15118 EVSE states map to the reported OCPP Connector Status.

EVSE State (ISO 15118)	OCPP Status	Notes
Init	Unavailable	System initialization or re-boot.
Wait	Available	Ready for a new session (CP State A).
SLAC	Preparing	Plugged in, matching physics.
Session, Authorization	Preparing	Protocol negotiation and Auth.
Params, Cable	Preparing	Parameter discovery and Cable Check.
PreCharge	Charging	Contactors closing, voltage matching.
Charge	Charging	Active energy transfer.
Charge (CurrentDemand=0)	SuspendedEV	EV is not drawing power.
Charge (limit=0)	SuspendedEVSE	EVSE is preventing power flow.
Stop	Finishing	Session ended, waiting for unplug.
(Any Error)	Faulted	Critical hardware or protocol failure.

Chapter 4

Configuration and Deployment

4.1 Command Line Interface (CLI)

The system is launched via `Application.py` with several arguments to define its operating mode:

- `-r, -role`: Specifies the mode of operation (EV or EVSE).
- `-i, -interface`: Defines the network interface connected to the Whitebeet module (e.g., `eth0`).
- `-c, -config`: Path to the EV configuration file (default: `ev.json`).
- `-ec, -evse-config`: Path to the EVSE configuration file (default: `evse.json`).
- `-auto`: (EVSE only) Enables automatic authorization for Plug & Charge.
- `-api-port`: Sets the port for the REST API status monitor.
- `-ocpp-url`: (EVSE only) WebSocket URL for the OCPP CSMS.
- `-ocpp-id`: (EVSE only) The Charge Point ID to Identify the station to the CSMS.

4.2 JSON Configuration

Behavioral parameters are externalized in JSON files to allow run-time customization without code changes.

4.2.1 EV Configuration (`ev.json`)

Defines the characteristics of the Electric Vehicle:

- **ev**: Supported protocols (ISO 15118-2, DIN), payment methods (EIM, PnC), and energy transfer modes.
- **battery**: Physical parameters including capacity, maximum voltage/current, and initial SOC.

4.2.2 EVSE Configuration (`evse.json`)

Defines the characteristics of the Charging Station:

- **charger**: Output limits (Max Voltage/Current/Power) and slew rates ($\Delta V, \Delta I$) for the simulation.

- **schedule:** Defines power delivery schedules (SASchedules) sent to the EV during the Charge Parameter Discovery phase.